上机反馈-第3周

处理程序输入

- 题目中通常包含多个测试数据，要分别计算并输出相应结果
- 许多同学的做法：
  - 假设有n行输入
  - 首先读进所有输入
  - 遍历所有输入，运行算法

```
# 读进所有输入
all_inputs = []
for i in range(n):
    all_inputs.append(...)

# 对每个输入计算结果并输出
for single_input in all_inputs:
    output = ...
```

- 更好的做法：
  - 定义一个函数处理单个测试数据
  - 读进一个输入，调用一次函数，并输出
  - 可读性更高，且易于定位和修改问题

```
def solution(single_input):
    return output

for i in range(n):
    single_input = input()
    print(solution(single_input))
```

未知行数的输入

- 有时，输入的行数是未知的，如何使用Python应对这种情形？
  - 输入行数未知时，困难在于无法确定何时停止输入
  - 对于文件输入，标准输入，可以使用EOF标志来表明输入流的结束

样例输入

```
sequence subsequence
person compression
VERDI vivaVittorioEmanueleReDiItalia
caseDoesMatter CaseDoesMatter
```

输入文件的EOF标志在这里

未知行数的输入

- 使用Python异常处理机制
  - input()函数遇到EOF标志时，产生EOFError。
  - 使用try-except语句捕获EOFError并停止输入
  - 在本地调试时，使用Ctrl+Z (Windows系统) 或 Ctrl+D (macOS/Unix系统)手动键入EOF标志符，表示输入结束

```python
# 在while循环中处理未知数量的输入
while True:
    try:
        s = input()
    except Exception:
        break
    print(">> " + s)
```

```
The first line of input
>> The first line of input
The second line of input
>> The second line of input
The last line of input
>> The last line of input
^Z
```

其他问题

- 这周讲完了顺序表与链表，是否意味着上机作业中一定需要使用这些内容？
  - 不严格要求。例如，不会因为在解法中没有使用特定数据结构而扣分
  - 课堂内容与上机作业的联系主要体现在算法设计思路上
- xxx题目的标准答案是怎么做的？
  - 算法可能有标准答案，但是实现方案通常没有标准答案
  - 可以参考他人的实现方式，但关键在于理解算法，关注算法复杂性
- 我的做法十分繁琐，怎么将其简化？
  - 我们会避免出极度繁琐的题目。所以这也是一个提示：当你的做法极度繁琐时，可以考虑更换一个思路。
  - 建议：在开始写代码之前，先写好注释
  - 例：Number Sequence，只需要计数模拟，不需要数学推导

关于上机作业

- 助教期望为你解决的问题：
  - 我对Python不熟悉，我该如何配置IDE / 运行调试我的程序？
  - 题目叙述不清。我应将其理解为xxx还是理解为yyy？
  - 我的xxx思路是否可行，是否有未考虑到的情形？
  - 我实现了xxx思路，结果 ME / TE，如何改进我的方法？
  - 我实现了xxx思路并通过了测评，是否有更好的方法？
  - 我无法理解程序报错的内容，我应该如何修正？
- 应该自己解决的问题
  - 我实现了xxx思路，结果 WA，我的代码有什么缺陷？
    - 助教无法总是为你提供解决方案
  - 为什么我的程序运行结果与期望不一致？
    - 先自行使用 print / 断点调试，尝试后再求助助教
  - Python中的xxx函数/特性是如何使用的
    - 先自行查阅，助教可以为你解释无法理解的内容
  - 题目的测试数据都有什么？
    - 助教也无法访问。并且，避免面向测试样例编程

Smart Questions

- 作为初学者，如何更好地向老师/助教/同学/网友提问？
  - 高效准确地描述你的问题
  - 为回答者的方便考虑
  - 展示你对此问题的思考与努力
- 参考《提问的智慧》这篇文章
  - https://lug.ustc.edu.cn/wiki/doc/smart-questions/
  - 最初为开源社区所作，流行于开源社区，是一个技术交流指南，也是一种沟通哲学
  - 其中的内容不局限于这门课程，适用于各领域的学习工作